The Journal of Open Source Software

PyStormTracker: A High-Performance Cyclone Tracker in Python

Albert M. W. Yau [1¶] **and Kevin I. Hodges** [2]

1 School of Marine and Atmospheric Sciences, Stony Brook University, Stony Brook, New York, USA **2** Department of Meteorology, University of Reading, Reading, United Kingdom ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

Storm-track activity is commonly described using Eulerian statistics of synoptic-scale variability and Lagrangian statistics derived from tracked weather systems (Chang et al., 2012; Hoskins & Hodges, 2002). The objective feature-tracking methodology developed by Hodges provides a general framework for identifying atmospheric features and constructing trajectories on the sphere, and has been used in a wide range of atmospheric applications (Hodges, 1994, 1995, 1999; Hodges et al., 2003). **PyStormTracker** (PST) is an open-source Python package for feature detection, cyclone tracking, trajectory comparison, and storm-track analysis.

PyStormTracker brings these tasks into the modern Python scientific ecosystem. It uses `xarray` for labeled meteorological data, `ducc0` for high-performance spherical numerical operations, NumPy and Numba for array-based computation, SciPy for established interpolation routines, and Dask or MPI for parallel execution. Three tracking families share the same packed trajectory representation: a lightweight local-extrema tracker descended from the original PyStormTracker implementation, a TRACK-compatible implementation of the Hodges feature-tracking methodology, and a HEALPix-topology variant for equal-area spherical grids (Górski et al., 2005). The same output model is used for trajectory comparison, variable sampling, serialization, and Eulerian and Lagrangian storm-track statistics.

Statement of need

Objective cyclone detection and tracking remains a difficult reproducibility and comparison problem. The IMILAST project compared multiple extratropical cyclone detection and tracking methods and found substantial differences among resulting cyclone climatologies (Neu et al., 2013). Intercomparison is scientifically valuable, but difficult when methods use different preprocessing, executables, coordinate conventions, output formats, and postprocessing. Reproducing an established method also requires implementation details such as the diagnostic field, spatial filtering, feature refinement, trajectory linking, treatment of missing points, and track filtering to be explicit. The tracked field and filtering are part of the scientific definition of a tracking experiment rather than interchangeable implementation details (Hoskins & Hodges, 2002).

The computational setting has also changed. NCEP/NCAR Reanalysis used a T62 atmospheric model (about 210 km), and widely used pressure-level products are provided on 2.5-degree grids (Kalnay et al., 1996; NOAA Physical Sciences Laboratory, 2026). ERA-Interim increased the native atmospheric resolution to T255/N128 (about 80 km), while ERA5 HRES uses T639/N320 (about 31 km) (European Centre for Medium-Range Weather Forecasts, 2019, 2020). Recent machine-learning weather systems have also explored native spherical meshes such as HEALPix (Karlbauer et al., 2024). Higher-resolution and non-regular grids make

spectral preprocessing, data movement, and intermediate-file I/O increasingly important parts of a tracking workflow.

PyStormTracker is intended for atmospheric scientists working with reanalysis, climate-model, and numerical-weather-prediction output who need reproducible cyclone tracking and storm-track diagnostics. It provides a common Python framework in which tracking algorithms share data access, preprocessing, trajectory representation, comparison tools, parallel execution, and downstream analysis. Tracking variables and filtering choices remain configurable and are recorded with the processing metadata rather than hidden in a separate preprocessing workflow. The package includes cyclone frequency, track frequency, cyclone amplitude, Accumulated Cyclone Activity (ACA), Accumulated Track Activity (ATA), Eulerian storm-track statistics, CORMAX, and EOF–CCA analysis.

State of the field

Hodges developed a general objective feature-tracking methodology during the 1990s, including feature-point identification, trajectory construction on the sphere, and constrained optimization of the resulting tracks (Hodges, 1994, 1995, 1999). The resulting TRACK system has since been used in many atmospheric applications, including studies of Northern and Southern Hemisphere storm tracks and comparisons of reanalysis datasets (Hodges et al., 2003; Hoskins & Hodges, 2002, 2005). Hoskins and Hodges (2002) compared tracking based on multiple meteorological fields and found lower-tropospheric relative vorticity particularly useful for synoptic systems because it is less dominated by the large-scale background, emphasizes smaller-scale features, and can identify developing systems earlier in their life cycle. They also noted that high-resolution vorticity can contain substantial small-scale structure, making spatial filtering or smoothing an important part of the tracking definition.

The scientific motivation underlying PyStormTracker predates the software. Chang (2014) showed, using a single tracking algorithm on 23 CMIP5 simulations, that projected Pacific winter cyclone changes can reverse sign depending on whether cyclones are defined from total sea-level pressure or from perturbations after removing a large-scale, low-frequency background. A later study compared sea-level-pressure-anomaly tracks, T5–T42-filtered 850-hPa relative-vorticity tracks, and 24-h pressure-change variance when assessing cyclone change and temperature impacts (Chang et al., 2016). These results motivate making background removal and filtering, the tracked variable, dataset provenance, and metric definition explicit.

Yau and Chang (2020) formalized the comparison of storm-track metrics against precipitation and strong-wind impacts using CORMAX and cross-validated EOF–CCA frameworks. They introduced ATA, combining cyclone track frequency and amplitude; among the Lagrangian definitions evaluated over Europe, ATA based on spatially filtered 850-hPa relative-vorticity maxima performed best for both impacts. The original PyStormTracker simple tracker was also used as an independent sensitivity test in that evaluation.

The PyStormTracker software effort began in 2015 with a simpler local-extrema and nearest-neighbor tracker, developed during NCAR SIParCS and presented at the 2016 AMS Annual Meeting (Yau et al., 2016). It was designed for scalability and extensibility, separating grid handling, detection, linking, and parallel execution. `mpi4py` distributed time ranges among ranks and combined partial track sets through a tree reduction, while comparison with the Hodges methodology was identified as a validation objective.

The present PyStormTracker development is being carried out in collaboration with Hodges to implement his tracking algorithms in the Python framework and validate their correspondence with TRACK. The tracking field is not restricted to mean sea-level pressure; the current implementation also supports relative-vorticity tracking, including the 850-hPa vorticity configuration commonly used in TRACK studies. Implementing the method in PyStormTracker allows the TRACK-compatible workflow to share xarray-based data handling, Dask/MPI execution, the common `Tracks` representation, comparison tools, and storm-track statistics

with alternative algorithms. TRACK 1.5.4 provides the implementation reference for parity testing, while PyStormTracker provides a Python framework for using, comparing, and extending these methods.

Software design

PyStormTracker separates **scientific algorithms**, **data representation**, and **execution policy**. Every tracker returns an immutable packed `Tracks` object containing aligned NumPy arrays for trajectory identifiers, offsets, times, coordinates, and variables. Individual tracks are lightweight views over these arrays. This common representation is the boundary used by serialization, trajectory comparison, variable sampling, and storm-track metrics.

Meteorological data handling is built around `xarray` (Hoyer & Hamman, 2017). File paths and xarray objects are normalized to labeled arrays before tracker-specific calculations. Preprocessing includes spectral filtering, tapering, regridding, and projection, with processing metadata recording the operations and parameters that actually occurred. This is scientifically consequential for feature tracking: for example, a T5–T42-filtered 850-hPa vorticity field defines a different feature population from unfiltered mean sea-level pressure. The data path is designed to minimize transformed intermediate files: in-memory inputs pass directly through preprocessing, detection, and refinement, while Dask-backed inputs keep preprocessing lazy and materialize complete spatial frames only as their tasks execute. Full-resolution fields are released before trajectory linking.

For spherical-harmonic preprocessing, PyStormTracker calls `ducc0` directly and implements the meteorological layer around its lower-level transforms: grid geometry, analysis and synthesis, spectral tapering and truncation, and regridding (Reinecke, 2020). This avoids depending on older atmospheric SHT stacks. NCL has been in maintenance mode since 2019, SPHEREPACK is among NCAR's classic Fortran libraries that are no longer under development, and the original `pyspharm` package's latest PyPI release is a source distribution from 2020 (National Center for Atmospheric Research, 2019, 2025; Whitaker, 2020). NumPy provides the core array representation, Numba compiles feature-detection, geometry, cost, and Modified Greedy Exchange hot loops, and SciPy supplies established interpolation and spline routines (Virtanen et al., 2020).

Parallel execution follows the scientific structure of the calculation. Independent time steps can be preprocessed, detected, and refined concurrently. Hodges and HEALPix trajectory optimization runs on overlapping temporal segments that are then spliced deterministically. The execution policy changes how independent work is scheduled without changing the tracking configuration or scientific definitions. Dask provides task-based execution on a multicore workstation or local cluster, `mpi4py` remains available for distributed-memory execution, and `ducc0` can use native threads within spherical transforms.

The same design improves high-resolution execution. In the controlled full-year 2024 ERA5 comparison, with source frames, T6–42 filtering, and tracking configuration held constant, changing reconstruction from T42 to F320 increased TRACK 1.5.4 wall time from 59.43 s to 1997.16 s (33.6 times), while PyStormTracker increased from 27.48 s to 57.38 s (2.09 times). These measurements are implementation- and machine-specific rather than an asymptotic claim about the Hodges algorithm; they show how the transformation and data path can dominate a native-resolution workflow.

Research impact statement

PyStormTracker has a public history dating to 2015–2016 and is now a tested, documented Python package distributed through PyPI, conda-forge, containers, and Zenodo (Yau, 2026). Its analysis layer implements the same classes of diagnostics motivated above: Eulerian and

Lagrangian storm-track statistics, ATA, CORMAX, and cross-validated EOF–CCA analysis. The original simple tracker has also been used in published sensitivity analysis (Yau & Chang, 2020).

The current implementation has quantitative comparison evidence against TRACK 1.5.4. In a 1,464-frame 2024 ERA5 mean-sea-level-pressure comparison using T6–42 filtering and the common RSPLICE population, the full-year one-to-one trajectory F1 is 99.7% for both F320-to-T42 and F320-to-F320 output grids, with median matched-point separations of 4.1 m and 4.9 m, respectively. On the recorded 16-core benchmark host, PyStormTracker was 2.16 times faster than TRACK for the full-year F320-to-T42 case and 34.81 times faster for F320-to-F320. Source-derived configurations and raw comparison records are maintained separately in the PyStormTracker-Validation repository.

AI usage disclosure

OpenAI generative-AI tools from the GPT-5.6 family, including Luna, Terra, and Sol configurations used through ChatGPT and Codex-style coding agents, assisted parts of the 2026 modernization. Assistance included repository exploration, alternative implementation proposals, refactoring, test scaffolding, code review, documentation editing, and manuscript drafting; the present draft was prepared with GPT-5.6 Sol. The corresponding author defined the software scope and core design decisions, selected which suggestions to accept, reviewed and edited AI-assisted changes, and validated them through tests, literature and primary-source review, TRACK source comparison, and reproducible benchmark/parity experiments. The authors remain responsible for the accuracy, originality, licensing, and scientific interpretation of the submitted material.

Acknowledgements

The original PyStormTracker project was supported through the National Center for Atmospheric Research 2015 SIParCS program and developed with Kevin Paul and John Dennis. The storm-track analysis framework implemented by the package grew from scientific collaboration with Edmund K. M. Chang. PyStormTracker also builds on the openly available TRACK source and the broader scientific Python ecosystem.

References

Chang, E. K. M. (2014). Impacts of background field removal on CMIP5 projected changes in pacific winter cyclone activity. *Journal of Geophysical Research: Atmospheres*, *119*(8), 4626–4639. https://doi.org/10.1002/2013JD020746

Chang, E. K. M., Guo, Y., & Xia, X. (2012). CMIP5 multimodel ensemble projection of storm track change under global warming. *Journal of Geophysical Research: Atmospheres*, *117*(D23), D23118. https://doi.org/10.1029/2012JD018578

Chang, E. K. M., Ma, C.-G., Zheng, C., & Yau, A. M. W. (2016). Observed and projected decrease in northern hemisphere extratropical cyclone activity in summer and its impacts on maximum temperature. *Geophysical Research Letters*, *43*(5), 2200–2208. https://doi.org/10.1002/2016GL068172

European Centre for Medium-Range Weather Forecasts. (2019). *ERA-Interim documentation: Spatial grid*. Copernicus Knowledge Base. https://confluence.ecmwf.int/pages/viewpage.action?pageId=155338777

European Centre for Medium-Range Weather Forecasts. (2020). *ERA5 data documentation: Spatial grid*. Copernicus Knowledge Base. https://confluence.ecmwf.int/pages/viewpage.

action?pageId=181128119

Górski, K. M., Hivon, E., Banday, A. J., Wandelt, B. D., Hansen, F. K., Reinecke, M., & Bartelmann, M. (2005). HEALPix: A framework for high-resolution discretization and fast analysis of data distributed on the sphere. *The Astrophysical Journal*, *622*(2), 759–771. https://doi.org/10.1086/427976

Hodges, K. I. (1994). A general method for tracking analysis and its application to meteorological data. *Monthly Weather Review*, *122*(11), 2573–2586. https://doi.org/10.1175/1520-0493(1994)122%3C2573:AGMFTA%3E2.0.CO;2

Hodges, K. I. (1995). Feature tracking on the unit sphere. *Monthly Weather Review*, *123*(12), 3458–3465. https://doi.org/10.1175/1520-0493(1995)123%3C3458:FTOTUS%3E2.0.CO;2

Hodges, K. I. (1999). Adaptive constraints for feature tracking. *Monthly Weather Review*, *127*(6), 1362–1373. https://doi.org/10.1175/1520-0493(1999)127%3C1362:ACFFT%3E2.0.CO;2

Hodges, K. I., Hoskins, B. J., Boyle, J., & Thorncroft, C. (2003). A comparison of recent reanalysis datasets using objective feature tracking: Storm tracks and tropical easterly waves. *Monthly Weather Review*, *131*(9), 2012–2037. https://doi.org/10.1175/1520-0493(2003)131%3C2012:ACORRD%3E2.0.CO;2

Hoskins, B. J., & Hodges, K. I. (2002). New perspectives on the northern hemisphere winter storm tracks. *Journal of the Atmospheric Sciences*, *59*(6), 1041–1061. https://doi.org/10.1175/1520-0469(2002)059%3C1041:NPOTNH%3E2.0.CO;2

Hoskins, B. J., & Hodges, K. I. (2005). A new perspective on southern hemisphere storm tracks. *Journal of Climate*, *18*(20), 4108–4129. https://doi.org/10.1175/JCLI3570.1

Hoyer, S., & Hamman, J. J. (2017). Xarray: N-D labeled arrays and datasets in python. *Journal of Open Research Software*, *5*(1), 10. https://doi.org/10.5334/jors.148

Kalnay, E., Kanamitsu, M., Kistler, R., Collins, W., Deaven, D., Gandin, L., Iredell, M., Saha, S., White, G., Woollen, J., Zhu, Y., Chelliah, M., Ebisuzaki, W., Higgins, W., Janowiak, J., Mo, K. C., Ropelewski, C., Wang, J., Leetmaa, A., … Joseph, D. (1996). The NCEP/NCAR 40-year reanalysis project. *Bulletin of the American Meteorological Society*, *77*(3), 437–472. https://doi.org/10.1175/1520-0477(1996)077%3C0437:TNYRP%3E2.0.CO;2

Karlbauer, M., Cresswell-Clay, N., Durran, D. R., Moreno, R. A., Kurth, T., Bonev, B., Brenowitz, N., & Butz, M. V. (2024). Advancing parsimonious deep learning weather prediction using the HEALPix mesh. *Journal of Advances in Modeling Earth Systems*, *16*(8). https://doi.org/10.1029/2023MS004021

National Center for Atmospheric Research. (2019). *NCL version 6.6.2 and maintenance mode*. NCAR Command Language documentation. https://www.ncl.ucar.edu/current_release.shtml

National Center for Atmospheric Research. (2025). *NCAR Classic Libraries for Geophysics*. NCAR HPC Documentation. https://ncar-hpc-docs.readthedocs.io/en/latest/environment-and-software/hpc-software/ncar-classic-libraries-for-geophysics/

Neu, U., Akperov, M. G., Bellenbaum, N., Benestad, R., Blender, R., Caballero, R., Cocozza, A., Dacre, H. F., Feng, Y., Fraedrich, K., Grieger, J., Gulev, S., Hanley, J., Hewson, T., Inatsu, M., Keay, K., Kew, S. F., Kindem, I., Leckebusch, G. C., … Wernli, H. (2013). IMILAST: A community effort to intercompare extratropical cyclone detection and tracking algorithms. *Bulletin of the American Meteorological Society*, *94*(4), 529–547. https://doi.org/10.1175/BAMS-D-11-00154.1

NOAA Physical Sciences Laboratory. (2026). *NCEP-NCAR reanalysis 1*. Dataset documentation. https://psl.noaa.gov/data/gridded/data.ncep.reanalysis.html

Reinecke, M. (2020). *DUCC: Distinctly useful code collection*. Astrophysics Source Code Library, record ascl:2008.023. https://ascl.net/2008.023

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … Mulbregt, P. van. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Whitaker, J. (2020). *Pyspharm: Python spherical harmonic transform module* (Version 1.0.9). https://pypi.org/project/pyspharm/

Yau, A. M. W. (2026). *PyStormTracker: A high-performance cyclone tracker in python*. Zenodo. https://doi.org/10.5281/zenodo.18764813

Yau, A. M. W., & Chang, E. K. M. (2020). Finding storm track activity metrics that are highly correlated with weather impacts. Part i: Frameworks for evaluation and accumulated track activity. *Journal of Climate*, *33*(23), 10169–10186. https://doi.org/10.1175/JCLI-D-20-0393.1

Yau, A. M. W., Paul, K., & Dennis, J. (2016). *PyStormTracker: A parallel object-oriented cyclone tracker in python*. 96th American Meteorological Society Annual Meeting, Sixth Symposium on Advances in Modeling and Analysis Using Python. https://doi.org/10.5281/zenodo.18868625